MACHINE LEARNING INTERVIEW PREPARATION

Classical NLP Foundations: Interview Questions & Answers

Comprehensive Classical NLP Foundations & Systems

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: Text Preprocessing, Vector Spaces, BM25, Embeddings, Language Models, RNN/LSTM/GRU, Drift Detection

Includes: Step-by-Step Numerical Hand Calculations, PyTorch Implementations, Production Telemetry

Module 10: NLP Fundamentals High-Frequency Interview Question Bank

This module provides detailed answers for the 40 standard and 10 advanced bonus interview questions covering NLP foundations, mathematical derivations, debugging procedures, and production systems design.

---

1. Conceptual Questions (1-12)

Question 1: What is NLP, and what are the major NLP tasks?

Short Interview Answer (30–60 seconds)

Natural Language Processing (NLP) is the domain of artificial intelligence focused on enabling computers to understand, interpret, and generate human language. The major tasks include text classification (sentiment analysis, spam detection), sequence tagging (Named Entity Recognition, Part-of-Speech tagging), sequence-to-sequence translation, and question answering.

Key Interview Points

- Semantic analysis
- Sequence labeling (NER/POS)
- Text generation & translation

Production Perspective & Trade-offs

In production, different tasks require different latency limits. Classification runs under strict <50ms budgets using sparse algorithms, whereas sequence-to-sequence generation using autoregressive decoders incurs high computational cost and latency.

Follow-up Questions

- **Follow-up**: *What makes NER harder than text classification?* -> NER requires token-level context mapping and boundary extraction, whereas classification aggregates sentence embeddings.

Common Mistakes

Confusing sequence labeling (NER) with sequence-to-sequence generation (translation).

Question 2: Explain a typical NLP pipeline from raw text to prediction.

Short Interview Answer (30–60 seconds)

The standard pipeline processes text through: raw input ingestion → text cleaning (normalization, regex) → tokenization (splitting into subwords) → text representation (mapping to sparse or dense embedding vectors) → model execution (running recurrent or self-attention layers) → prediction output (generating logits) → metric evaluation.

Key Interview Points

- Preprocessing & normalization
- Subword tokenization
- Vector embeddings
- Inference execution

Production Perspective & Trade-offs

Ensure preprocessing steps are identical between training and inference to prevent vocabulary mapping drift. Real-time inference pipelines often cache embedding weights to reduce database lookup overhead.

Follow-up Questions

- **Follow-up**: *Where does training-serving skew occur in this pipeline?* -> It usually happens when text normalization rules (like lowercasing or Unicode normalization) differ between training and serving code.

Common Mistakes

- Ignoring Unicode normalization, which leads to split character representations.

Question 3: What is the difference between stemming and lemmatization?

Short Interview Answer (30–60 seconds)

Stemming is a rule-based heuristic that truncates word endings (suffixes) to find a base form. Lemmatization uses morphological lookup tables and part-of-speech (POS) tags to resolve words to their canonical dictionary base form (lemma).

Key Interview Points

- Heuristic truncation (Stemming)
- Morphological dictionary lookup (Lemmatization)
- POS tagging dependency

Production Perspective & Trade-offs

- **Stemming**: Extremely fast, low memory usage, but can produce non-words (e.g. `"studies"` → `"studi"`).
- **Lemmatization**: Grammatically accurate, but has high latency due to POS tagging and dictionary lookups.

Follow-up Questions

- **Follow-up**: *Which is preferred for web search indexing?* -> Lemmatization is preferred because it maps words accurately to real dictionary terms, improving search query recall.

Common Mistakes

- Assuming stemming is always superior because of its lower execution latency.

Question 4: Why is tokenization necessary?

Short Interview Answer (30–60 seconds)

Computers cannot process unstructured strings directly. Tokenization decomposes text streams into discrete lexical chunks (words, subwords, or characters) that can be mapped to unique indices in a model's vocabulary table.

Key Interview Points

- Lexical splitting
- Vocabulary mapping
- Index lookup tables

Production Perspective & Trade-offs

Choosing the tokenization boundary directly impacts model parameter count. A very large vocabulary increases the size of the final projection layer, whereas a small character-level vocabulary increases input sequence lengths, raising attention computation costs.

Follow-up Questions

- **Follow-up**: *Can we build an NLP system without a tokenizer?* -> Yes, by processing raw bytes directly (e.g. Byte-level models), but this increases training times and sequence lengths.

Common Mistakes

- Treating tokenization as a simple string split on whitespace, which fails to handle punctuation and clitics (like `"don't"`).

Question 5: Compare character, word, and subword tokenization.

Short Interview Answer (30–60 seconds)

Character tokenization splits text into letters, resulting in small vocabularies but long sequence arrays. Word tokenization splits on word boundaries, keeping sequence lengths short but creating massive vocabularies with high out-of-vocabulary (OOV) rates. Subword tokenization (BPE/WordPiece) strikes a balance by splitting common roots while decomposing rare words into sub-components.

Key Interview Points

- Vocabulary size vs. Sequence length
- Out-of-Vocabulary (OOV) rates
- Computational balance

Production Perspective & Trade-offs

Subword tokenizers (like WordPiece) are standard in production LLMs because they keep vocabulary tables compact ($32\text{k}–256\text{k}$) while preventing OOV errors during user queries.

Follow-up Questions

- **Follow-up**: *Which tokenizer type is most robust against typos?* -> Character-level or subword tokenizers, as they can decompose misspelled words into known character sequences.

Common Mistakes

- Believing character-level tokenization is more computationally efficient (it actually increases sequence length, raising attention computational cost quadratically).

Question 6: Why did modern NLP move from word tokenization to subword tokenization?

Short Interview Answer (30–60 seconds)

Word-level tokenization struggles with vocabulary explosion ($|V| > 1,000,000$) and high OOV rates. Subword tokenization represents text using a smaller vocabulary of root prefixes and suffixes, allowing models to process new or misspelled words without crashing or generating `<unk>` tokens.

Key Interview Points

- Vocabulary size limits
- Out-of-Vocabulary (OOV) mitigation
- Subword decomposition

Production Perspective & Trade-offs

Subword vocabularies fit easily into GPU memory, freeing up VRAM for longer sequence lengths and larger model hidden dimensions.

Follow-up Questions

- **Follow-up**: *How does BPE handle numerical data?* -> It often splits numbers into individual digits or byte sequences, preventing the model from needing a separate token for every integer.

Common Mistakes

- Believing subwords are manually defined by linguists (they are learned statistically from training data).

Question 7: What are Out-of-Vocabulary (OOV) words, and how can they be handled?

Short Interview Answer (30–60 seconds)

OOV words are tokens encountered during inference that were not present in the training vocabulary. They can be handled by mapping them to a fallback `<unk>` token, utilizing subword tokenizers (BPE) to decompose them, or using character-level models like FastText.

Key Interview Points

- Unknown tokens

- Byte-fallback vocabularies
- FastText n-gram recovery

Production Perspective & Trade-offs

Using `<unk>` degrades model accuracy because the model loses the semantic content of the unknown token. FastText handles this by generating vectors from subword n-grams.

Follow-up Questions

- **Follow-up**: *Why does BERT not return OOV errors?* -> BERT uses WordPiece tokenization, which decomposes unknown words down to individual characters if needed.

Common Mistakes

- Believing that increasing training vocabulary size to include every possible word completely resolves OOV issues.

Question 8: Explain the Distributional Hypothesis.

Short Interview Answer (30–60 seconds)

The Distributional Hypothesis states that words that occur in similar contexts share semantic meaning. This forms the foundation for dense word representations: models learn embeddings by predicting words based on their neighbors.

Key Interview Points

- Contextual semantics
- Word co-occurrences
- Vector projection spaces

Production Perspective & Trade-offs

This hypothesis allows models to learn representations from unstructured text, eliminating the need for expensive manual semantic labeling.

Follow-up Questions

- **Follow-up**: *What is a limitation of this hypothesis?* -> Antonyms (like `"hot"` and `"cold"`) often appear in identical contexts, leading to similar vector representations despite having opposite meanings.

Common Mistakes

- Assuming the hypothesis requires dictionary-defined semantic tags to compute vector similarities.

Question 9: Why do sparse text representations fail to capture semantic similarity?

Short Interview Answer (30–60 seconds)

Sparse representations (One-Hot, Bag of Words) treat each word as an independent, orthogonal dimension. Consequently, the dot product between different words (e.g. `"cat"` and `"feline"`) is 0, capturing zero semantic overlap.

Key Interview Points

- Vector orthogonality
- Sparse dimensionality
- Zero-product overlap

Production Perspective & Trade-offs

Sparse representations scale with vocabulary size ($|V|$), leading to high memory footprint and data sparsity during downstream training.

Follow-up Questions

- **Follow-up**: *How does Cosine Similarity help evaluate sparse vectors?* -> It measures overlap on shared coordinates, but fails if documents use synonyms.

Common Mistakes

- Assuming TF-IDF captures word similarities (it only matches exact token strings).

Question 10: Compare static embeddings and contextual embeddings.

Short Interview Answer (30–60 seconds)

Static embeddings (Word2Vec, GloVe) assign a single fixed vector to each word, regardless of context. Contextual embeddings (BERT, GPT) generate dynamic vectors for each token based on the surrounding sentence context.

Key Interview Points

- Polysemy resolution
- Static vocabulary tables
- Dynamic encoder projections

Production Perspective & Trade-offs

- **Static**: Fast, constant $O(1)$ memory lookup, but cannot resolve word meanings dynamically.
- **Contextual**: High computation cost (requires transformer forward passes), but resolves word context (e.g., `"bank"` in `"river bank"` vs. `"money bank"`).

Follow-up Questions

- **Follow-up**: *Can static embeddings resolve part-of-speech tags?* -> No, because the word vector remains identical regardless of whether it is used as a noun or a verb.

Common Mistakes

- Assuming static embeddings are obsolete (they remain useful for low-latency similarity searches).

Question 11: Why were RNNs introduced after Bag-of-Words and TF-IDF?

Short Interview Answer (30–60 seconds)

Bag-of-Words and TF-IDF discard word order and syntax. RNNs process tokens sequentially, updating a hidden state to capture temporal dependencies and word order.

Key Interview Points

- Sequence order preservation
- Variable length handling
- Hidden state memory

Production Perspective & Trade-offs

RNNs process text sequentially, creating a training bottleneck that prevents parallel GPU execution.

Follow-up Questions

- **Follow-up**: *How does RNN state size impact VRAM usage?* -> Larger hidden states require storing larger intermediate vectors during Backpropagation Through Time (BPTT).

Common Mistakes

- Believing RNNs can process tokens in parallel like Transformers.

Question 12: Why were Transformers able to replace RNNs?

Short Interview Answer (30–60 seconds)

Transformers replaced RNNs by utilizing self-attention, which processes all tokens in parallel and reduces token-to-token path lengths to $O(1)$. This allows for faster training speeds and better scaling on long sequences.

Key Interview Points

- Parallel training path
- Long-range dependencies ($O(1)$)
- Self-attention projection

Production Perspective & Trade-offs

Transformers scale training efficiently but require quadratic $O(L^2)$ memory during inference due to the attention matrix computation.

Follow-up Questions

- **Follow-up**: *Why do Transformers need positional encodings?* -> Self-attention is permutation-invariant; without positional encodings, it would treat sentences as unordered bags of words.

Common Mistakes

- Assuming Transformers require less memory than RNNs (their attention matrix is memory-intensive).

---

2. Mathematical Questions (13-22)

Question 13: Derive the TF-IDF equation and compute TF-IDF scores for a small corpus.

Short Interview Answer (30–60 seconds)

TF-IDF (Term Frequency-Inverse Document Frequency) measures a token's information density relative to a corpus. The mathematical formula is:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

$$\text{IDF}(t, D) = \ln\left(\frac{1 + N}{1 + \text{DF}(t)}\right) + 1$$

Where $\text{TF}(t, d)$ is the frequency of term t in document d, N is the total documents in corpus D, and $\text{DF}(t)$ is the document frequency of term t.

Technical Intuition & Detailed Hand-Calculation

Consider a micro-corpus of $N = 2$ documents:

- d_1: `"cat mat"`
- d_2: `"mat rug"`
  Vocabulary: `["cat", "mat", "rug"]` (vocabulary size $|V| = 3$).

Step 1: Calculate Document Frequencies (DF) and Smooth IDF for each term:

- **"cat"**: $\text{DF}(\text{"cat"}) = 1$ (appears only in d_1)

$$\text{IDF}(\text{"cat"}) = \ln\left(\frac{1 + 2}{1 + 1}\right) + 1 = \ln(1.5) + 1 \approx 0.4055 + 1 = 1.4055$$

- **"mat"**: $\text{DF}(\text{"mat"}) = 2$ (appears in d_1 and d_2)

$$\text{IDF}(\text{"mat"}) = \ln\left(\frac{1 + 2}{1 + 2}\right) + 1 = \ln(1.0) + 1 = 0.0000 + 1 = 1.0000$$

- **"rug"**: $\text{DF}(\text{"rug"}) = 1$ (appears only in d_2)

$$\text{IDF}(\text{"rug"}) = \ln\left(\frac{1 + 2}{1 + 1}\right) + 1 = \ln(1.5) + 1 \approx 0.4055 + 1 = 1.4055$$

Step 2: Compute Raw TF-IDF Vector for d_1 (frequencies: $\text{TF}(\text{"cat"}) = 1$, $\text{TF}(\text{"mat"}) = 1$, $\text{TF}(\text{"rug"}) = 0$):

- $\text{TF-IDF}(\text{"cat"}, d_1) = 1 \times 1.4055 = 1.4055$
- $\text{TF-IDF}(\text{"mat"}, d_1) = 1 \times 1.0000 = 1.0000$
- $\text{TF-IDF}(\text{"rug"}, d_1) = 0 \times 1.4055 = 0.0000$

$$\mathbf{v}_{d_1} = [1.4055,\ 1.0000,\ 0.0000]^T$$

Step 3: Apply L_2 Normalization to prevent document length bias:

$$\|\mathbf{v}_{d_1}\|_2 = \sqrt{1.4055^2 + 1.0000^2 + 0.0^2} = \sqrt{1.9754 + 1.0000} = \sqrt{2.9754} \approx 1.7249$$

$$\mathbf{v}_{d_1,\text{norm}} = \left[\frac{1.4055}{1.7249}, \frac{1.0000}{1.7249}, 0.0\right]^T \approx [0.8148,\ 0.5797,\ 0.0]^T$$

Production Perspective & Trade-offs

- **Length Normalization**: Essential because longer documents naturally repeat words, inflating TF scores. L_2 normalization maps vectors onto a unit hypersphere, allowing Cosine Similarity to measure angles rather than absolute vector lengths.
- **Sparse Storage**: Large corpora yield $|V| > 100,000$, resulting in memory-heavy matrices. Production systems store representations in coordinate (COO) or compressed sparse row (CSR) formats to minimize storage overhead.

Follow-up Questions

- **Follow-up**: *How does a document frequency of 0 impact IDF?* -> Smooth IDF adds 1 to both the numerator and denominator, preventing division-by-zero errors.

Common Mistakes

- Forgetting to apply smooth factors or normalization, leading to document length bias.

Question 14: Compute cosine similarity between two TF-IDF vectors.

Short Interview Answer (30–60 seconds)

Cosine similarity measures the angle between two vectors:

$$\text{CosineSimilarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2}$$

If vectors are pre-normalized to unit length, this simplifies to the dot product.

Technical Intuition & Complexity

Given vectors:

- $\mathbf{a} = [0.8, 0.6, 0.0]$

- $\mathbf{b} = [0.0, 0.6, 0.8]$

$$\mathbf{a} \cdot \mathbf{b} = (0.8 \times 0) + (0.6 \times 0.6) + (0 \times 0.8) = 0.36$$

Production Perspective & Trade-offs

Dot products run efficiently on modern hardware using BLAS libraries, making cosine similarity comparisons fast in production.

Follow-up Questions

- **Follow-up**: *What is the cosine similarity of orthogonal vectors?* -> 0, indicating no shared token dimensions.

Common Mistakes

- Neglecting to normalize vectors, which biases similarity scores towards longer documents.

Question 15: Using the chain rule, derive the probability of a sentence in an N-gram language model.

Short Interview Answer (30–60 seconds)

By the chain rule, the joint probability of a sequence is:

$$P(w_1, \ldots, w_m) = \prod_{i=1}^{m} P(w_i \mid w_1, \ldots, w_{i-1})$$

An N-gram model simplifies this using the Markov assumption, limiting the context window to the preceding $N - 1$ words:

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-N+1}, \ldots, w_{i-1})$$

Key Interview Points

- Joint probability factorization
- Markov context windows
- Conditional sequencing

Production Perspective & Trade-offs

Larger context orders N capture more dependencies but increase table memory footprints exponentially ($O(|V|^N)$).

Follow-up Questions

- **Follow-up**: *Why do we use log probabilities in evaluations?* -> Multiplying small values leads to numerical underflow; summing log probabilities keeps calculations stable.

Common Mistakes

- Forgetting the boundary markers (like `<s>` and `</s>`) when calculating sequence probabilities.

Question 16: Explain Maximum Likelihood Estimation (MLE) for N-gram language models.

Short Interview Answer (30–60 seconds)

MLE estimates sequence transitions by counting occurrences in a training corpus:

$$P_{\text{MLE}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Where $C(w_{i-1}, w_i)$ is the bigram co-occurrence count.

Technical Intuition & Complexity

If the bigram `"sat on"` appears 10 times and `"sat"` appears 100 times, the MLE probability $P(\text{"on"} \mid \text{"sat"}) = 10/100 = 0.1$.

Production Perspective & Trade-offs

- **Time Complexity**: $O(1)$ constant time lookup if using precomputed n-gram tables.
- **Memory Complexity**: $O(|V|^N)$ space.

Follow-up Questions

- **Follow-up**: *What happens if a prefix is unseen during training?* -> The denominator becomes 0, crashing the calculation unless smoothing is applied.

Common Mistakes

- Assuming MLE generalizes well to unseen text without smoothing.

Question 17: Why is Laplace smoothing required? Compute smoothed probabilities for a simple example.

Short Interview Answer (30–60 seconds)

In Maximum Likelihood Estimation (MLE), any word transition unseen in the training set receives a probability score of 0. Due to the chain rule of probability, a single zero probability collapses the entire joint sequence probability to 0. Laplace smoothing (add-one smoothing) solves this by adding 1 to all counts, shifting probability mass from seen events to unseen events:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

Where $C(w_{i-1}, w_i)$ is the bigram count, $C(w_{i-1})$ is the unigram history count, and $|V|$ is the vocabulary size.

Technical Intuition & Detailed Hand-Calculation

Let's train a bigram language model on the micro-corpus:
`"the cat sat on the mat"` (contains 6 tokens).
Vocabulary: `["the", "cat", "sat", "on", "mat"]` (vocabulary size $|V| = 5$).

Step 1: Track counts for unigrams and bigrams:

- $C(\text{"the"}) = 2$
- $C(\text{"the"}, \text{"cat"}) = 1$
- $C(\text{"the"}, \text{"mat"}) = 1$
- $C(\text{"the"}, \text{"sat"}) = 0$ (unseen transition)

Step 2: Compute smoothed transition probabilities:

- **Seen transition** $P_{\text{Laplace}}(\text{"cat"} \mid \text{"the"})$:

$$P_{\text{Laplace}}(\text{"cat"} \mid \text{"the"}) = \frac{C(\text{"the"}, \text{"cat"}) + 1}{C(\text{"the"}) + |V|} = \frac{1+1}{2+5} = \frac{2}{7} \approx 0.2857$$

- **Unseen transition** $P_{\text{Laplace}}(\text{"sat"} \mid \text{"the"})$:

$$P_{\text{Laplace}}(\text{"sat"} \mid \text{"the"}) = \frac{C(\text{"the"}, \text{"sat"}) + 1}{C(\text{"the"}) + |V|} = \frac{0+1}{2+5} = \frac{1}{7} \approx 0.1429$$

Notice that the sum of probabilities across all possible next tokens for the history `"the"` remains normalized:

$$\sum_{w \in V} P(w \mid \text{"the"}) = \frac{2}{7}(\text{for "cat"}) + \frac{2}{7}(\text{for "mat"}) + \frac{1}{7}(\text{for "sat"}) + \frac{1}{7}(\text{for "on"}) + \frac{1}{7}(\text{for '}$$

Production Perspective & Trade-offs

- **The $|V|$ Bottleneck**: While Laplace smoothing ensures mathematical validity, adding 1 to all unseen transitions allocates a massive amount of probability mass to the long tail of rare or unseen words in large vocabularies (e.g. $|V| = 50,000$), significantly degrading the probability of common, natural transitions.
- **Production Solution**: Modern NLP systems rely on Absolute Discounting or Kneser-Ney smoothing, which discount a fixed value d from seen counts and distribute it proportionally based on how likely a word is to complete unseen context (continuation probability).

Follow-up Questions

- **Follow-up**: *How does Kneser-Ney smoothing improve on Laplace?* -> Kneser-Ney uses absolute discounting and a continuation probability to estimate how likely a word is to complete an unseen context.

Common Mistakes

- Forgetting to add vocabulary size $|V|$ to the denominator, which violates probability normalization rules.

Question 18: What is Perplexity? Derive its mathematical formulation and explain its intuition.

Short Interview Answer (30–60 seconds)

Perplexity (PPL) measures how well a probability model predicts a sample. Intuitively, it represents the average branching factor—the number of equally likely words the model is choosing from at each step. Mathematically, it is the exponentiated cross-entropy loss of the sequence:

$$\text{PPL}(W) = P(w_1, w_2, \ldots, w_m)^{-\frac{1}{m}} = e^{\mathcal{L}_{\text{CE}}}$$

Where m is the sequence length, $P(w_1, \ldots, w_m)$ is the sequence joint probability, and $\mathcal{L}_{\text{CE}}$ is the average cross-entropy loss per token.

Technical Intuition & Detailed Hand-Calculation

Let's derive perplexity from cross-entropy and compute it for a tiny 3-token sequence:

1. Mathematical Derivation:

The cross-entropy loss per token for a sequence W of length m is:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{m}\sum_{i=1}^{m} \ln P(w_i \mid w_{1:i-1}) = -\frac{1}{m}\ln P(w_1, \ldots, w_m)$$

Exponentiating both sides:

$$e^{\mathcal{L}_{\text{CE}}} = e^{-\frac{1}{m}\ln P(W)} = \left(e^{\ln P(W)}\right)^{-\frac{1}{m}} = P(W)^{-\frac{1}{m}} = \text{PPL}(W)$$

2. Step-by-Step Hand-Calculation:

Suppose our bigram model processes a test sequence $W = (w_1, w_2, w_3)$ with the following conditional probabilities:

- $P(w_1) = 0.50$
- $P(w_2 \mid w_1) = 0.20$
- $P(w_3 \mid w_2) = 0.40$
- **Step 1: Compute Joint Probability:**

  $$P(W) = P(w_1) \times P(w_2 \mid w_1) \times P(w_3 \mid w_2) = 0.50 \times 0.20 \times 0.40 = 0.0400$$

- **Step 2: Calculate Average Cross-Entropy Loss ($\mathcal{L}_{\text{CE}}$):**

  $$\mathcal{L}_{\text{CE}} = -\frac{1}{3}\ln(0.0400) \approx -\frac{1}{3} \times (-3.2189) = 1.0730$$

- **Step 3: Calculate Perplexity:**

  $$\text{PPL}(W) = e^{1.0730} = 0.0400^{-\frac{1}{3}} = 25^{\frac{1}{3}} \approx 2.9240$$

Findings & Interpretation:

The perplexity of 2.9240 means that at each token prediction step, the model was as confused as if it had to choose uniformly from ≈ 2.92 candidate words. A lower perplexity indicates a more confident model.

Production Perspective & Trade-offs

- **Comparison Limits**: Perplexity can only be used to compare models that share the exact same tokenizer and vocabulary size. A model with a vocabulary size $|V| = 1,000$ will inherently have a lower baseline perplexity than a model with $|V| = 50,000$, even if the latter is semantically superior, because it is choosing from fewer alternatives.

- **Out of Vocabulary (OOV)**: If a tokenizer fails to handle OOV tokens, a single unseen token can make $P(w_i) = 0$, causing PPL to explode to infinity, requiring clipping or smoothing.

Follow-up Questions

- **Follow-up**: *How does vocabulary size impact perplexity comparisons?* -> Models with smaller vocabularies inherently yield lower perplexity scores because they choose from fewer options.

Common Mistakes

- Comparing perplexity scores across models that use different tokenizers or vocabularies.

Question 19: Explain how CBOW and Skip-gram learn word embeddings.

Short Interview Answer (30–60 seconds)

Word2Vec trains static embeddings using local context predictions:

- **CBOW**: Predicts a target word given its surrounding context words.
- **Skip-gram**: Predicts context words given a target word.

Technical Intuition & Complexity

- **CBOW**: Context vectors are averaged into a single vector $\mathbf{h}$ to predict the target.
- **Skip-gram**: Runs multiple predictions per target word, making it slower to train but yielding better representations for rare tokens.

Production Perspective & Trade-offs

- **CBOW**: Fast to train, but averages out context details.
- **Skip-gram**: Slower to train, but captures context details and rare tokens well.

Follow-up Questions

- **Follow-up**: *What optimization algorithms speed up Word2Vec?* -> Negative Sampling and Hierarchical Softmax.

Common Mistakes

- Assuming Word2Vec requires labeled training data (it is self-supervised).

Question 20: Why does Negative Sampling make Word2Vec training faster?

Short Interview Answer (30–60 seconds)

Standard multiclass Softmax requires computing a normalization sum over the entire vocabulary $|V|$ (often $> 100,000$ tokens) in the denominator for every single training step, which is computationally prohibitive. Skip-gram with Negative Sampling (SGNS) reformulates the task as a binary logistic regression game. For each target-context pair, the model optimizes a binary classifier to distinguish the true target-context pair from K randomly drawn "noise" (negative) samples, reducing computing complexity from $O(|V|)$ to $O(K)$.

The SGNS loss function is:

$$\mathcal{L}_{\text{SGNS}} = -\ln \sigma(\mathbf{v}_w \cdot \mathbf{v}'_{w_c}) - \sum_{i=1}^{K} \ln \sigma(-\mathbf{v}_w \cdot \mathbf{v}'_{w_i})$$

Technical Intuition & Detailed Hand-Calculation

Let's compute the negative sampling loss for a single step with a key target word and $K = 1$ negative sample:

- Target word: `"cat"` (context embedding vector $\mathbf{v}_{\text{cat}} = [0.10,\ 0.80,\ -0.20]^T$)
- Positive context word: `"sat"` (target embedding vector $\mathbf{v}'_{\text{sat}} = [0.20,\ 0.90,\ 0.10]^T$)
- Negative noise word: `"refrigerator"` (target embedding vector $\mathbf{v}'_{\text{refrig}} = [-0.90,\ 0.10,\ 0.80]^T$)

Step 1: Compute Dot Products:

- Positive pair dot product:

$$\mathbf{v}_{\text{cat}} \cdot \mathbf{v}'_{\text{sat}} = (0.10 \times 0.20) + (0.80 \times 0.90) + (-0.20 \times 0.10) = 0.0200 + 0.7200 - 0.0$$

- Negative pair dot product:

$$\mathbf{v}_{\text{cat}} \cdot \mathbf{v}'_{\text{refrig}} = (0.10 \times -0.90) + (0.80 \times 0.10) + (-0.20 \times 0.80) = -0.0900 + 0.0800$$

Step 2: Compute Logistic Sigmoid Probabilities ($\sigma(x) = \frac{1}{1+e^{-x}}$):

- Positive pair probability:

$$\sigma(0.7200) = \frac{1}{1 + e^{-0.7200}} \approx \frac{1}{1 + 0.4868} = 0.6726$$

- Negative pair probability (note the negative sign $-\mathbf{v} \cdot \mathbf{v}'$):

$$\sigma(-(-0.1700)) = \sigma(0.1700) = \frac{1}{1 + e^{-0.1700}} \approx \frac{1}{1 + 0.8437} = 0.5424$$

Step 3: Calculate the SGNS Loss:

$$\mathcal{L}_{\text{SGNS}} = -\ln(0.6726) - \ln(0.5424) \approx 0.3966 + 0.6117 = 1.0083$$

Findings & Interpretation:

The loss score is 1.0083. During backpropagation, the gradients will adjust only the vectors for `"cat"`, `"sat"`, and `"refrigerator"`. The other $|V| - 3$ word vectors are completely untouched during this step, which is why it runs orders of magnitude faster.

Production Perspective & Trade-offs

- **Time Complexity**: Reduces weight-update calculations from $O(|V|)$ to $O(K)$.
- **Negative Sampling Distribution**: Negatives are sampled from a smoothed unigram distribution:

$$P_n(w) \propto U(w)^{0.75}$$

  Raising unigram frequency $U(w)$ to the power of 0.75 boosts the probability of sampling rare words (e.g. if $U(w_1) = 0.99$ and $U(w_2) = 0.01$, the ratio shifts from $99:1$ to $\approx 32:1$), preventing the model from over-optimizing on common stop words (like `"the"` or `"is"`).

Follow-up Questions

- **Follow-up**: *What is the optimal value for K?* -> Typically 5–20 for small datasets, and 2–5 for large corpora.

Common Mistakes

- Believing negative sampling updates all vocabulary weights at each step.

Question 21: Explain mathematically why RNNs suffer from vanishing gradients.

Short Interview Answer (30–60 seconds)

During backpropagation through time (BPTT), gradients are scaled by the recurrent weight matrix product:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \text{diag}(1 - h_k^2) W_{hh}^T$$

If the largest eigenvalue of W_{hh} is less than 1, multiplying this matrix repeatedly over a long sequence causes the gradient to shrink exponentially to 0.

Key Interview Points

- Backpropagation through time (BPTT)
- Jacobian chain product
- Spectral radius $\rho(W_{hh}) < 1$

Production Perspective & Trade-offs

Vanishing gradients prevent standard RNNs from learning long-range dependencies, requiring the use of LSTMs or GRUs.

Follow-up Questions

- **Follow-up**: *How does gradient clipping resolve exploding gradients?* -> If the gradient norm exceeds a threshold, it is scaled down: $\mathbf{g} \leftarrow \mathbf{g} \cdot \frac{g_{\max}}{\|\mathbf{g}\|}$.

Common Mistakes

- Confusing vanishing gradients with dead ReLU units.

Question 22: Explain how LSTM's cell state mitigates the vanishing gradient problem.

Short Interview Answer (30–60 seconds)

Standard RNNs propagate gradients by multiplying the error vector repeatedly by the recurrent hidden-to-hidden weight matrix $\mathbf{W}_{hh}$ at each backpropagation step. This multiplicative chain leads to vanishing gradients if the eigenvalues of $\mathbf{W}_{hh}$ are less than 1. In contrast, LSTMs propagate error gradients through an additive **Constant Error Carousel (CEC)** located in the cell state $\mathbf{C}_t$. The cell state update equation is linear:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

Since the update is additive, the derivative $\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}}$ has a direct linear pathway (scaled by forget gate $\mathbf{f}_t$), bypassing recurrent weight matrix multiplication.

Technical Intuition & Detailed Derivation

To see how LSTM mitigates vanishing gradients, let's derive the gradient pathway back from cell state $\mathbf{C}_t$ to $\mathbf{C}_{t-1}$:

1. Derive the Cell State Jacobian:

The cell state update is:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

Differentiating $\mathbf{C}_t$ with respect to $\mathbf{C}_{t-1}$ yields:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \mathbf{f}_t + \mathbf{C}_{t-1} \odot \frac{\partial \mathbf{f}_t}{\partial \mathbf{C}_{t-1}} + \tilde{\mathbf{C}}_t \odot \frac{\partial \mathbf{i}_t}{\partial \mathbf{C}_{t-1}} + \mathbf{i}_t \odot \frac{\partial \tilde{\mathbf{C}}_t}{\partial \mathbf{C}_{t-1}}$$

2. The CEC Shortcut:

If we assume that the forget gate is fully open ($\mathbf{f}_t \approx \mathbf{1}$) and the hidden states are structured such that the gate derivatives (the latter three terms in the sum) are close to 0:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} \approx \mathbf{f}_t \approx \mathbf{1}$$

3. Propagating over $T - t$ steps:

$$\frac{\partial \mathbf{C}_T}{\partial \mathbf{C}_t} = \prod_{k=t+1}^{T} \frac{\partial \mathbf{C}_k}{\partial \mathbf{C}_{k-1}} \approx \prod_{k=t+1}^{T} \mathbf{f}_k$$

If $\mathbf{f}_k \approx 1$ (the forget gate remains open), the error gradient propagates back across arbitrarily long sequence lengths without decaying. This additive, gated shortcut is called the **Constant Error Carousel (CEC)**.

Production Perspective & Trade-offs

- **Forget Gate Calibration**: The forget gate $\mathbf{f}_t$ is crucial. If the model is not trained well and $\mathbf{f}_t \to 0$, gradients will vanish just as in standard RNNs.
- **Compute Overhead**: The gating mechanism requires four separate linear layers per cell ($\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t, \tilde{\mathbf{C}}_t$), quadrupling the parameter count and computational budget compared to standard RNNs. In high-throughput production settings, engineers often use GRUs (Gate Recurrent Units) which merge cell and hidden states to save 25% on parameter size.

Follow-up Questions

- **Follow-up**: *What happens if the forget gate is always 0?* -> The model discards all historical cell state information at each step, behaving like a standard feedforward network.

Common Mistakes

- Believing LSTMs completely eliminate vanishing gradients (they can still vanish if the forget gate outputs 0).

---

3. Production & Engineering Questions (23-30)

Question 23: What challenges arise when deploying NLP models to production?

Short Interview Answer (30–60 seconds)

Production challenges include: managing GPU/CPU latency budgets (especially for autoregressive text generation), handling vocabulary drift, maintaining preprocessing consistency between training and serving, and managing deployment costs.

Key Interview Points

- Latency budgets
- Vocabulary drift
- Model quantization

Production Perspective & Trade-offs

Deploying models requires balancing latency and accuracy. Quantization reduces VRAM footprints but can degrade accuracy on edge cases.

Follow-up Questions

- **Follow-up**: *How does model pruning affect execution speed?* -> Pruning zeroes out weights to create sparse matrices, which can bypass calculations on hardware that supports sparse operations.

Common Mistakes

- Assuming research accuracies translate directly to production without latency optimization.

Question 24: How do you handle vocabulary drift in production systems?

Short Interview Answer (30–60 seconds)

Vocabulary drift occurs when user inputs introduce new words or symbols (e.g. slang, emojis) not present in the model's vocabulary. We handle this by using byte-fallback tokenizers, retraining models on live data, and maintaining dynamic lookup dictionary expansions.

Key Interview Points

- Byte-fallback tokenization
- Retraining loops
- Dynamic dictionary expansions

Production Perspective & Trade-offs

Byte-fallback tokenizers decompose unknown words down to raw bytes, preventing `<unk>` errors at the cost of slightly longer sequence lengths.

Follow-up Questions

- **Follow-up**: *How does vocabulary size impact model serving cost?* -> A larger vocabulary increases the classification layer's size, increasing GPU VRAM usage.

Common Mistakes

- Relying on manual dictionary additions, which fails to scale in dynamic environments.

Question 25: What is training-serving skew? Why is it problematic?

Short Interview Answer (30–60 seconds)

Training-serving skew occurs when the preprocessing pipeline, data distribution, or environment features differ between the training phase and the production serving layer, leading to model degradation.

Key Interview Points

- Preprocessing consistency
- Feature drift
- Pipeline alignment

Production Perspective & Trade-offs

To prevent skew, wrap tokenizers and preprocessing code into a shared, version-controlled library used by both training and serving pipelines.

Follow-up Questions

- **Follow-up**: *How does Unicode decomposition cause training-serving skew?* -> If the serving tokenizer composition doesn't match the training setup, words split differently, mapping to incorrect token indices.

Common Mistakes

- Using separate codebases (e.g., Python for training, C++ for serving) without strict testing against index outputs.

Question 26: Explain Data Drift versus Concept Drift with examples.

Short Interview Answer (30–60 seconds)

- **Data Drift**: The input data distribution changes, but input-to-label relationships remain the same.
- **Concept Drift**: The mapping relationship between inputs and labels shifts over time.

Key Interview Points

- Covariate shift
- Semantic label drift
- Population monitoring

Production Perspective & Trade-offs

- **Data Drift Example**: A sentiment classifier trained on news articles processes social media comments containing slang.
- **Concept Drift Example**: The term `"viral"` shifts from representing a negative healthcare quarantine tag (2019) to a positive marketing indicator (2021).

Follow-up Questions

- **Follow-up**: *How do you monitor for concept drift?* -> By tracking performance metrics (like F1 score or accuracy) on labeled production data over time.

Common Mistakes

- Retraining models on raw inputs to resolve concept drift without verifying updated label mappings.

Question 27: When would you choose batch inference over real-time inference?

Short Interview Answer (30–60 seconds)

Choose **Batch Inference** when throughput is the main priority and real-time response latency is not required (e.g. running classification sweeps over nightly logs). Choose **Real-Time Inference** when processing interactive streaming user inputs with strict latency limits (e.g. search query auto-completion).

Key Interview Points

- High throughput vs. Low latency
- GPU utilization profiles
- Cost optimization

Production Perspective & Trade-offs

Batch inference maximizes GPU utilization by processing large chunks of data in parallel, lowering cost per token. Real-time inference processes single inputs, which can leave GPUs underutilized.

Follow-up Questions

- **Follow-up**: *How does dynamic batching optimize real-time inference?* -> It groups incoming concurrent user requests into a single batch at the server level, increasing throughput.

Common Mistakes

- Deploying real-time servers for tasks that could be run as cheaper batch pipelines.

Question 28: How do quantization and pruning reduce inference cost?

Short Interview Answer (30–60 seconds)

- **Quantization**: Converts model weights from float32 (32-bit) to lower-precision formats like float16 or int8, reducing VRAM footprint and memory bandwidth limits.

- **Pruning**: Zeroes out non-essential parameters, creating sparse matrices that can speed up inference.

Key Interview Points

- Precision conversion (int8)
- Weight sparsity
- VRAM footprint compression

Production Perspective & Trade-offs

Quantization reduces VRAM usage by up to 75%, allowing models to run on cheaper hardware, but can degrade accuracy on edge cases.

Follow-up Questions

- **Follow-up**: *What is Post-Training Quantization (PTQ) vs. Quantization-Aware Training (QAT)? ->* PTQ quantizes weights after training, while QAT models precision loss during training, yielding better accuracy.

Common Mistakes

- Assuming pruning always speeds up models (it requires hardware that supports sparse operations to yield speed gains).

Question 29: What preprocessing inconsistencies can lead to degraded model performance?

Short Interview Answer (30–60 seconds)

Inconsistencies include: using different lowercase rules, different regex string cleanups, different Unicode normalizations (NFC vs. NFD), or different subword vocabularies between training and serving.

Key Interview Points

- Token normalization mismatch
- Unicode normalizations
- Pipeline testing

Production Perspective & Trade-offs

A single preprocessing mismatch (like missing punctuation cleaning) can cause the tokenizer to split words differently, mapping them to incorrect vectors.

Follow-up Questions

- **Follow-up**: *How does regex cleaning impact latency?* -> Complex regex lookaheads run on CPU and can become a bottleneck in high-throughput pipelines.

Common Mistakes

- Assuming standard tokenizers handle raw, uncleaned text consistently without normalizations.

Question 30: What monitoring metrics would you track for an NLP model in production?

Short Interview Answer (30–60 seconds)

Track: **Systems Metrics** (inference latency, GPU VRAM usage, error rates), **Data Metrics** (out-of-vocabulary token rates, input length distributions, data drift metrics), and **Performance Metrics** (prediction confidence distributions, feedback classifications).

Key Interview Points

- GPU/VRAM health
- Data drift distributions
- OOV token rates

Production Perspective & Trade-offs

Set up automated alarms on OOV rates. A spike in OOV tokens indicates data drift, signaling that the model needs retraining.

Follow-up Questions

- **Follow-up**: *How do you measure drift on text embeddings?* -> By tracking cosine distance distributions between production embeddings and training reference vectors over time.

Common Mistakes

- Monitoring only systems metrics (like CPU load) while neglecting data drift and model accuracy decay.

---

4. Debugging & Evaluation Questions (31-35)

Question 31: Why can BLEU and ROUGE give misleading evaluation results?

Short Interview Answer (30–60 seconds)

BLEU and ROUGE measure exact n-gram overlaps. They suffer from semantic blind spots: they score synonyms as zero matches (e.g. `"feline"` vs `"cat"`) and can assign high scores to grammatically similar but logically opposite generations (such as negations).

Key Interview Points

- N-gram overlap limits
- Synonym blind spots
- Negation mismatch

Production Perspective & Trade-offs

Using only BLEU/ROUGE can lead to deploying models that output grammatically correct but factually incorrect summaries. Use semantic metrics like BERTScore alongside human validation loops.

Follow-up Questions

- **Follow-up**: *How does METEOR improve on BLEU?* -> METEOR incorporates stem matching and synonyms using dictionary databases (like WordNet) to compute alignments.

Common Mistakes

- Believing a high BLEU score guarantees factual correctness in text generation.

Question 32: When would you prefer BERTScore over BLEU?

Short Interview Answer (30–60 seconds)

Prefer **BERTScore** when evaluating semantic similarity, paraphrasing, or tasks where synonyms (e.g. `"feline"` instead of `"cat"`) are acceptable, because exact-match n-gram metrics (BLEU, ROUGE) assign a score of 0 to synonyms. Prefer **BLEU** or **ROUGE** for machine translation regression testing where exact vocabulary matches or speed are critical. BERTScore computes greedy cosine alignments over contextual embeddings from a pre-trained language model, capturing semantic shifts that overlap metrics miss.

Technical Intuition & Detailed Hand-Calculation

Let's compute BERTScore greedy alignment scores for:

- Candidate (c): `"A feline rested on the rug"` (Length $c = 6$)
- Reference (r): `"The cat sat on the mat"` (Length $r = 6$)

Suppose our contextual embedding model generates vectors for each token, and the maximum cosine similarity scores between candidate and reference tokens are:

- `"A"` matches best with `"The"` (Similarity = 0.12)
- `"feline"` matches best with `"cat"` (Similarity = 0.88)
- `"rested"` matches best with `"sat"` (Similarity = 0.82)
- `"on"` matches best with `"on"` (Similarity = 0.95)
- `"the"` matches best with `"the"` (Similarity = 0.98)
- `"rug"` matches best with `"mat"` (Similarity = 0.85)

1. Calculate BERTScore Recall (R_{BERT}):

Aligns each reference token to the best matching candidate token, normalized by reference length:

$$R_{\text{BERT}} = \frac{1}{|r|} \sum_{w_j \in r} \max_{w_i \in c} \mathbf{E}(w_i)^T \mathbf{E}(w_j) = \frac{0.12 + 0.88 + 0.82 + 0.95 + 0.98 + 0.85}{6} = \frac{4.60}{6} \approx$$

2. Calculate BERTScore Precision (P_{BERT}):

Aligns each candidate token to the best matching reference token, normalized by candidate length:

$$P_{\text{BERT}} = \frac{1}{|c|} \sum_{w_i \in c} \max_{w_j \in r} \mathbf{E}(w_i)^T \mathbf{E}(w_j) = \frac{0.12 + 0.88 + 0.82 + 0.95 + 0.98 + 0.85}{6} = \frac{4.60}{6} \approx$$

3. Calculate BERTScore F1 (F_{BERT}):

$$F_{\text{BERT}} = 2 \times \frac{P_{\text{BERT}} \times R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \approx 2 \times \frac{0.7667 \times 0.7667}{0.7667 + 0.7667} \approx 0.7667$$

Findings & Interpretation:

While BLEU-2 would penalize this candidate heavily due to lack of exact word matches (BLEU unigram overlap would fail on `"feline"` vs `"cat"` and `"rug"` vs `"mat"`), BERTScore detects the semantic equivalence of `"feline" \leftrightarrow "cat"` (0.88) and `"rug" \leftrightarrow "mat"` (0.85), yielding a high F1 score of 0.7667, capturing synonym alignments.

Production Perspective & Trade-offs

- **Inference Latency**: BLEU is a string overlap check running in $O(L_c \cdot L_r)$ on CPU (taking $< 0.1\text{ms}$). BERTScore requires running a transformer encoder forward-pass on GPU to extract token embeddings, increasing computing cost and latency by orders of magnitude, making it expensive for real-time model evaluation APIs.
- **Model Bias**: BERTScore scores depend on the underlying embedding model. A model biased against certain structures can skew evaluation scores, requiring careful selection of the encoder (e.g. RoBERTa-large).

Follow-up Questions

- **Follow-up**: *How does BERTScore calculate greedy alignments?* -> It computes cosine similarities between all candidate and reference token embeddings, matching each word to its highest scoring semantic counterpart.

Common Mistakes

- Neglecting the compute cost of running BERTScore over large evaluation datasets.

Question 33: How would you debug an NLP classifier with poor precision but high recall?

Short Interview Answer (30–60 seconds)

Poor precision but high recall means the classifier is over-predicting the positive class, resulting in many false positives. To debug, adjust the prediction threshold (increase the probability cutoff for the positive class), analyze the confusion matrix, and address class imbalances in the training data.

Key Interview Points

- High False Positives
- Positive threshold adjustments
- Imbalanced training data

Production Perspective & Trade-offs

In spam filtering, high recall with low precision means the model catches all spam but filters out valid emails. Adjust prediction thresholds to prioritize precision.

Follow-up Questions

- **Follow-up**: *How does F1 score help evaluate this trade-off?* -> The F1 score is the harmonic mean of precision and recall, helping you find a balance between the two metrics.

Common Mistakes

- Retraining the model from scratch without first adjusting prediction thresholds.

Question 34: How would you diagnose exploding gradients during RNN training?

Short Interview Answer (30–60 seconds)

Exploding gradients show up as training loss spiking to `NaN`, weight parameters diverging, or gradients showing extremely large norms during backpropagation. Diagnose this by monitoring gradient norms at each step. Fix it by applying gradient clipping or reducing the learning rate.

Key Interview Points

- Loss spikes to NaN
- Gradient norm monitors
- Gradient clipping

Production Perspective & Trade-offs

Gradient clipping prevents exploding gradients but does not address vanishing gradients. Use LSTMs or GRUs to handle vanishing gradients.

Follow-up Questions

- **Follow-up**: *What max norm value is typically used for clipping?* -> A max norm threshold of 1.0 is standard in deep learning pipelines.

Common Mistakes

- Assuming `NaN` loss is always caused by division-by-zero errors in the data rather than exploding gradients.

Question 35: How would you perform error analysis for an NLP system?

Short Interview Answer (30–60 seconds)

Perform error analysis by: logging classification outputs, filtering for predictions where ground-truth and prediction labels mismatch, manually inspecting a representative sample (e.g. 100-200 errors), categorizing errors into groups (e.g., tokenization errors, typos, class bias), and implementing targeted training data updates or preprocessing rules to resolve them.

Key Interview Points

- Mismatch logs audits
- Manual error labeling
- System retrain patches

Production Perspective & Trade-offs

Error analysis helps you identify specific preprocessing bugs or data drift patterns, allowing you to implement targeted fixes instead of blindly retraining the model.

Follow-up Questions

- **Follow-up**: *How does class imbalance impact error analysis?* -> It often causes the model to over-predict the majority class, resulting in high rates of false negatives for the minority class.

Common Mistakes

- Relying only on aggregate metrics (like accuracy) without auditing individual error logs.

---

5. System Design & Applied Questions (36-40)

Question 36: Design a spam email classifier for millions of users.

Short Interview Answer (30–60 seconds)

The system uses: a preprocessing pipeline to normalize HTML and emails $\rightarrow$ Feature Hashing (to maintain a zero-memory vocabulary footprint) $\rightarrow$ a fast classifier model (like logistic regression or Naïve Bayes) for initial filtering $\rightarrow$ a secondary model (like an LSTM or transformer) for low-confidence queries.

Key Interview Points

- High throughput pipeline

- Feature Hashing trick
- Multi-tier classification

Production Perspective & Trade-offs

- **High throughput**: Use Feature Hashing to handle high volume without storing massive lookup dictionaries.
- **Low latency**: Filter obvious spam early using cheap models, routing only ambiguous emails to resource-intensive classification models.
- **Drift**: Monitor OOV rates and user spam flags to detect vocabulary drift.

Follow-up Questions

- **Follow-up**: *How do you handle attachments?* -> Extract text from attachments using parser libraries, or route them through separate file scanner pipelines.

Common Mistakes

- Proposing heavy, slow transformer models for the entire email volume, which incurs high compute cost and latency.

Question 37: Design an autocomplete/search suggestion system.

Short Interview Answer (30–60 seconds)

The system uses: a trie structure populated with query probabilities. For the current prefix, the system looks up the top K completions with the highest MLE probabilities. Smooth prediction probabilities using Katz backoff or interpolation to handle rare or unseen prefixes.

Key Interview Points

- Trie prefix lookups
- MLE probabilities
- Katz backoff smoothing

Production Perspective & Trade-offs

- **Low Latency**: Suggestion lookups must run in <10ms. Store the prefix trie in memory (e.g. using Redis), caching frequent suggestions.
- **Storage**: Prune n-grams with count frequencies < 5 to compress trie memory size.

Follow-up Questions

- **Follow-up**: *How do you personalize suggestions?* -> By weighting the trie path probabilities based on the user's historical search categories.

Common Mistakes

- Querying SQL databases directly for auto-complete lookups, which is too slow for real-time systems.

Question 38: Design a sentiment analysis pipeline for social media.

Short Interview Answer (30–60 seconds)

The system uses: a preprocessing step that retains emojis and punctuation (cues for sentiment) $\rightarrow$ subword tokenization (BPE) $\rightarrow$ a bidirectional classifier model $\rightarrow$ metric logging. Monitor for data drift using population stability checks.

Key Interview Points

- Emoji/slang preservation
- Bidirectional context
- Data drift monitoring

Production Perspective & Trade-offs

Social media text features high rates of typos, slang, and emojis. Retaining emojis and using subword tokenization is critical to capture sentiment cues. Monitor data drift closely, as vocabulary shifts quickly on social platforms.

Follow-up Questions

- **Follow-up**: *Why use a bidirectional model?* -> It captures negation words (like `"not"`) that appear before or after the target word, resolving local context.

Common Mistakes

- Discarding emojis and punctuation during preprocessing, which removes critical sentiment signals.

Question 39: Design an FAQ chatbot using classical NLP techniques (without LLMs).

Short Interview Answer (30–60 seconds)

The system matches queries against a database of FAQs: pre-process user query → compute TF-IDF representation vector → query the FAQ database using BM25 → select the answer associated with the highest similarity score.

Key Interview Points

- BM25 keyword matching
- Similarity ranking
- Preprocessing normalizations

Production Perspective & Trade-offs

- **Advantages**: Fast, deterministic, runs on CPU with minimal hosting costs.
- **Disadvantages**: Cannot handle paraphrasing or queries that use synonyms.
- **Fix**: Use Sentence-BERT to generate semantic embeddings for queries, combining BM25 keyword matching and semantic search.

Follow-up Questions

- **Follow-up**: *How do you handle spelling errors?* -> Apply spelling correction algorithms or use subword n-grams (FastText) to compute similarities.

Common Mistakes

- Proposing heavy generative models when simple query-matching retrieval systems are sufficient.

Question 40: Given an NLP application, how would you decide between: TF-IDF, Word2Vec, FastText, LSTM, Transformer?

Short Interview Answer (30–60 seconds)

Select based on accuracy requirements, latency budgets, and sequence complexity:

- **TF-IDF**: Best for simple keyword searches or classification on static text with tight compute budgets.
- **Word2Vec**: Best for semantic similarity lookups on static vocabularies.

- **FastText**: Best when handling out-of-vocabulary (OOV) tokens, typos, or morphologically rich languages.
- **LSTM**: Best for processing sequence structures when context length is moderate and training resources are limited.
- **Transformer**: Best when accuracy is the main priority and you have the compute resources to support parallel training and long contexts.

Key Interview Points

- Latency vs. Accuracy
- OOV handling needs
- Compute resources

Production Perspective & Trade-offs

In production, start with a cheap model (TF-IDF or Word2Vec) to establish a baseline. Upgrade to LSTMs or Transformers only if the accuracy gains justify the higher latency and hosting costs.

Follow-up Questions

- **Follow-up**: *Which model handles multilingual text best?* -> FastText or Transformers, as they decompose text into subwords, reducing the size of multilingual vocabulary tables.

Common Mistakes

- Defaulting to complex Transformer models without evaluating simpler baselines first.

---

6. Advanced Questions (41-50)

Question 41: Why is BM25 generally better than TF-IDF for retrieval?

Short Interview Answer (30–60 seconds)

BM25 improves on TF-IDF by scaling term frequency non-linearly to prevent saturation and penalizing long documents that contain terms simply due to size:

- **TF-IDF**: Scales term frequency linearly, allowing a term repeated 100 times to dominate the score.
- **BM25**: Uses a saturation parameter k_1 to bound the score contribution of any single term, and normalizes by document length using parameter b.

Key Interview Points

- Term frequency saturation (k_1)
- Document length normalization (b)
- Sub-linear scaling

Production Perspective & Trade-offs

BM25 is standard in search engines (like Elasticsearch) because it prevents long documents containing query terms from dominating search rankings.

Follow-up Questions

- **Follow-up**: *What is the effect of setting parameter b to 0?* -> Length normalization is deactivated; document length has no impact on similarity scoring.

Common Mistakes

- Believing BM25 requires deep neural network evaluations (it is a keyword count algorithm).

Question 42: Why does FastText outperform Word2Vec for rare words?

Short Interview Answer (30–60 seconds)

Word2Vec learns independent vectors for each word, meaning rare words have poorly trained embeddings due to insufficient context samples. FastText represents words as a bag of character n-grams, allowing rare words to inherit vector representations from shared subwords, improving semantic alignment.

Key Interview Points

- Character n-gram embeddings
- Shared root semantics
- Typo resilience

Production Perspective & Trade-offs

FastText is resilient against typos and out-of-vocabulary (OOV) terms, making it useful in production environments with noisy user inputs.

Follow-up Questions

- **Follow-up**: *Does FastText require more memory than Word2Vec?* -> Yes, because it must store embeddings for both words and character n-grams.

Common Mistakes

- Assuming FastText is as slow as context-dependent transformer models (it is still a static lookup table model).

Question 43: Why are bidirectional RNNs unsuitable for autoregressive text generation?

Short Interview Answer (30–60 seconds)

Autoregressive generation generates text sequentially (token by token). Bidirectional RNNs require the entire input sequence to compute backward hidden states. Consequently, they cannot generate text because future tokens are not yet known.

Key Interview Points

- Autoregressive generation
- Future context dependency
- Causal masking

Production Perspective & Trade-offs

For generation tasks, use causal decoder models (which use causal masking to prevent the model from looking ahead) to ensure step-by-step token generation.

Follow-up Questions

- **Follow-up**: *Where are bidirectional models useful?* -> In sequence tagging (NER, POS tagging) and text representation (BERT), where the entire input sentence is available.

Common Mistakes

- Proposing bidirectional models (like BERT) for text generation tasks.

Question 44: How does teacher forcing affect Seq2Seq training?

Short Interview Answer (30–60 seconds)

During training, Teacher Forcing feeds the ground-truth target tokens as input to the decoder instead of the model's own previous predictions. This prevents early prediction errors from propagating through the sequence, stabilizing and speeding up early training.

Key Interview Points

- Ground-truth target inputs
- Training sequence alignment
- Exposure bias

Production Perspective & Trade-offs

- **Advantages**: Speeds up convergence during training.
- **Disadvantages**: Introduces **Exposure Bias** because the model never encounters its own errors during training, leading to generation errors during production inference.

Follow-up Questions

- **Follow-up**: *How do you mitigate exposure bias?* -> Using scheduled sampling, where you gradually transition from ground-truth inputs to the model's own predictions as training progresses.

Common Mistakes

- Using teacher forcing during inference (when ground-truth targets are not available).

Question 45: Explain greedy decoding versus beam search.

Short Interview Answer (30–60 seconds)

- **Greedy Search**: Selects the single most likely token at each step ($y_t = \arg\max P(y \mid y_{
- **Beam Search**: Explores multiple paths ($O(L \cdot B)$), keeping a running set of the B most likely sequences. It improves generation quality at the cost of higher latency and memory usage.

Key Interview Points

- Local vs. Global optimization
- Beam width parameter (B)
- Latency trade-offs

Production Perspective & Trade-offs

In production, large beam sizes (e.g. $B > 10$) increase latency and memory usage. A beam size of 2–5 is typically preferred to balance speed and quality.

Follow-up Questions

- **Follow-up**: *Why apply length normalization in beam search?* -> Without normalization, beam search favors shorter generations because multiplying probabilities yields smaller values as sequence length increases.

Common Mistakes

- Believing beam search guaranteed to find the globally optimal sequence (it is still a heuristic search).

Question 46: What is exposure bias in Seq2Seq models?

Short Interview Answer (30–60 seconds)

Exposure bias occurs when a model is trained using teacher forcing (receiving ground-truth inputs) but is evaluated during inference by feeding its own predictions back as input. Early prediction errors propagate, causing the model to generate low-quality text.

Key Interview Points

- Training-inference skew
- Error propagation
- Scheduled sampling

Production Perspective & Trade-offs

Exposure bias can cause generation models to output repetitive or irrelevant text when generating long sequences in production.

Follow-up Questions

- **Follow-up**: *How does Reinforcement Learning (RLHF) address exposure bias?* -> By optimizing the model based on the quality of the entire generated sequence, aligning training with inference behavior.

Common Mistakes

- Assuming exposure bias is a problem with the training data rather than a training-inference skew.

Question 47: Why is perplexity not always a good evaluation metric?

Short Interview Answer (30–60 seconds)

Perplexity only measures how well the model predicts the next token in the test set. It does not measure semantic correctness, factual accuracy, or logical consistency. A model can generate grammatically correct nonsense and still receive a low perplexity score.

Key Interview Points

- Next-token prediction focus
- Factual blind spots
- Tokenizer dependency

Production Perspective & Trade-offs

Perplexity is highly dependent on tokenizer vocabulary. You cannot compare perplexity scores across models that use different tokenizers.

Follow-up Questions

- **Follow-up**: *What metrics evaluate summarization quality better than perplexity?* -> BERTScore or LLM-as-a-Judge, which evaluate semantic correctness and factual consistency.

Common Mistakes

- Comparing perplexity scores between models that use different subword tokenizers.

Question 48: How does Byte Pair Encoding (BPE) differ from WordPiece and Unigram Language Models?

Short Interview Answer (30–60 seconds)

- **BPE**: A bottom-up tokenizer that iteratively merges the most frequent adjacent character pairs.
- **WordPiece**: A bottom-up tokenizer that merges pairs that maximize corpus likelihood, using a scoring ratio.
- **Unigram**: A top-down tokenizer that starts with a large vocabulary and prunes tokens that have the lowest contribution to corpus likelihood.

Key Interview Points

- Bottom-up vs. Top-down

- Frequency-based (BPE) vs. Likelihood-based (WordPiece)
- Entropy-based pruning (Unigram)

Production Perspective & Trade-offs

Unigram tokenizers offer more flexible subword segmentations, which can improve translation performance in multilingual models.

Follow-up Questions

- **Follow-up**: *Which tokenizer is used in BERT?* -> WordPiece is used in BERT, while BPE is used in GPT models.

Common Mistakes

- Assuming all subword tokenizers build vocabularies using the same merging criteria.

Question 49: Why are contextual embeddings superior to static embeddings?

Short Interview Answer (30–60 seconds)

Contextual embeddings generate dynamic vectors based on the surrounding sentence context, resolving word ambiguity (polysemy). Static embeddings assign a single fixed vector to each word, failing to handle polysemy (e.g. `"bank"` in `"river bank"` vs. `"money bank"`).

Key Interview Points

- Polysemy resolution
- Dynamic vector mapping
- Contextual semantics

Production Perspective & Trade-offs

Contextual embeddings require running transformer layers, which increases serving costs and latency compared to static embedding lookups.

Follow-up Questions

- **Follow-up**: *How does BERT construct contextual embeddings?* -> By processing tokens through multiple self-attention layers that update representations based on all other words in the sentence.

Common Mistakes

- Assuming static embeddings are obsolete (they remain useful for low-latency similarity searches).

Question 50: What are the computational complexity differences between RNNs and self-attention?

Short Interview Answer (30–60 seconds)

- **RNN**: Sequential updates scale as $O(L \cdot d^2)$ time complexity. Path length between distant tokens scales as $O(L)$ sequential steps, preventing parallel training.
- **Self-Attention**: Matrix operations scale as $O(L^2 \cdot d)$ time and memory complexity. Path length between any two tokens is $O(1)$, enabling parallel training.

Key Interview Points

- $O(L \cdot d^2)$ sequential vs. $O(L^2 \cdot d)$ parallel
- $O(L)$ path length vs. $O(1)$ path length
- Quadratic sequence bottleneck ($O(L^2)$)

Production Perspective & Trade-offs

While self-attention enables fast, parallel training, its quadratic memory cost ($O(L^2)$) makes serving long sequences resource-intensive.

Follow-up Questions

- **Follow-up**: *At what sequence length L does self-attention compute cost exceed RNNs?* -> When sequence length L exceeds the hidden dimension d ($L > d$), self-attention becomes more computationally expensive than RNNs.

Common Mistakes

- Assuming self-attention is always more computationally efficient than RNNs (it is only more efficient during training due to parallelization).